

MASTER CURSOR PROMPT – STRIPE PAYMENTS & FINANCE SYSTEM

Context & Objective

You are working inside an existing production-grade system with a ****separated frontend dashboard, client-side application, and backend****. You must carefully read and respect the architecture, patterns, and conventions of the following repositories: • Frontend (Client / Events / Organizers): symmetrical-fortnight-clone • Admin Dashboard Frontend: dashboard-byfoods • Backend (Core System): expert-spork-backend Your task is to ****design and implement a complete Stripe-based financial system**** with a strong focus on: security, Stripe policy compliance, smooth UX, scalability, auditability, and revenue generation. This is a large system; do NOT take shortcuts or introduce breaking changes.

Global Rules (Very Important)

- Do NOT implement Stripe payments for Restaurants.
- Restaurants should only receive a database-level update in the user/customer table to track revenue.
- Stripe payments MUST be implemented for Event and Tour roles only.
- Stripe onboarding must follow the latest Stripe Connect & KYC policies.
- All flows must be reversible, auditable, and safe from partial failures.

1. Roles & Financial Capabilities

Define and enforce the following behavior: • Customer/User: - Can book events/tours - Can pay via Stripe Checkout or Payment Intents - Can receive booking & refund notifications • Admin (Event / Tour Organizer): - Must complete Stripe Connect onboarding - Must complete KYC via dashboard stepper - Can receive payouts only after approval - Can view revenue, balances, payouts, refunds - Can request payouts (not auto-release) • Super Admin: - Reviews Stripe status of admins - Approves or rejects payout requests - Can send reminder emails to admins with incomplete KYC - Has full financial visibility • Restaurant: - NO Stripe integration - Only revenue fields updated in database

2. Stripe Architecture (Backend)

Implement Stripe using ****Stripe Connect (Express or Custom as appropriate)****. Backend responsibilities: • Create Stripe Connected Accounts for Event/Tour admins • Store Stripe account IDs securely • Handle webhooks for: - account.updated - checkout.session.completed - payment_intent.succeeded - charge.refunded - payout.paid / payout.failed • Maintain idempotency keys for all Stripe writes • Maintain a full transaction ledger table • Separate platform fees from organizer earnings

3. Client-Side Payments (Users)

On successful event/tour booking: • Create PaymentIntent / Checkout Session • Lock booking inventory before payment confirmation • Confirm payment success via webhook (NOT frontend only) • Send notifications: - To Admin (booking received) - To Customer (booking confirmed) • Store: -

gross amount - platform fee - admin net revenue

4. Refunds (Partial & Full)

Refund system requirements: • Admin can request refunds from dashboard • Super Admin can approve refunds if required • Stripe refund must sync back to DB • Ledger must reflect: - original payment - refunded amount - adjusted balances • Customer & Admin both receive refund notifications

5. Admin Dashboard – KYC & Onboarding

Implement a ****Stepper-based onboarding UX**** aligned with Stripe policies: Steps: 1. Basic business info 2. Personal identity verification 3. Bank account details 4. Stripe verification status 5. Activation confirmation Rules: • Disable payouts until Stripe verification = complete • Show clear blockers and missing requirements • Real-time status sync using Stripe account.updated webhook • Push notification + email reminder for incomplete KYC

6. Payout Flow (Admins)

- Admin requests payout from dashboard • Request enters PENDING_SUPER_ADMIN_APPROVAL
- Super Admin reviews and approves • Backend triggers Stripe payout • Final status synced back to dashboard • All payout attempts must be logged

7. Notifications System

Trigger notifications for: • Successful booking (Admin + Customer) • Refund processed (Admin + Customer) • KYC incomplete (Admin) • Payout approved / rejected (Admin) Use: • Push notifications (dashboard) • Email notifications • Notification persistence in DB

8. Locations & Discovery

When Admins add locations: • Persist structured location data (lat/lng, city, country) • Expose location on: - Event detail pages - Organizer profile pages • Ensure location filters work efficiently

9. UX & Financial Safety Principles

- Never rely solely on frontend payment confirmation • All balances derived from backend ledger • Prevent double payouts & race conditions • Clear empty/error states in dashboard • Finance pages must feel calm, clear, and trustworthy • Every financial action must be traceable

Final Instruction

Proceed step-by-step. Respect existing code patterns. Refactor where necessary, but do not rewrite blindly. This system must be production-ready, Stripe-compliant, and scalable.